



**NANYANG  
TECHNOLOGICAL  
UNIVERSITY**  
SINGAPORE

# Discrete Mathematics

## MH1812

**Topic 10.1 - Graph Theory I**  
**Dr. Wang Huaxiong**

# Topic Overview

# What's in store...

**I**

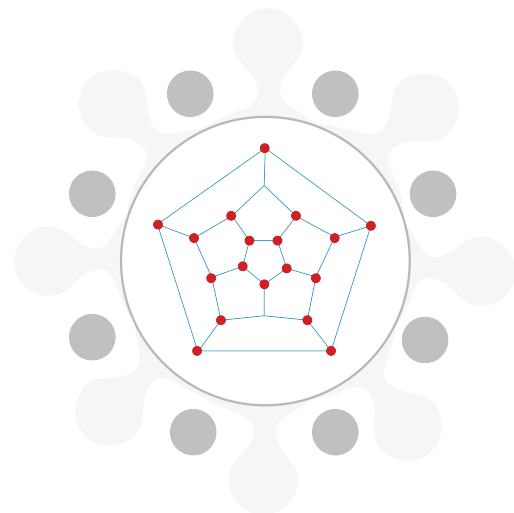
ntroduction to Graphs

**T**

ypes of Graphs

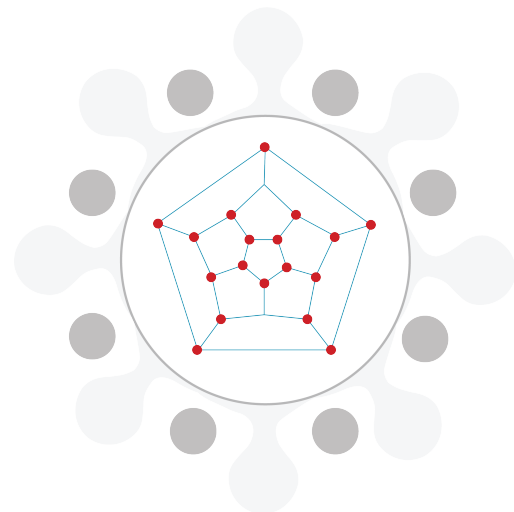
**E**

uler Theorem



# By the end of this lesson, you should be able to...

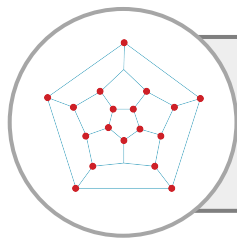
- Explain what is a graph.
- Explain the difference between the simple graph, multigraph and directed (multi) graph.
- Explain the concepts of the Euler path and circuit.
- Use the Euler theorem in graph theory.





# Introduction to Graphs

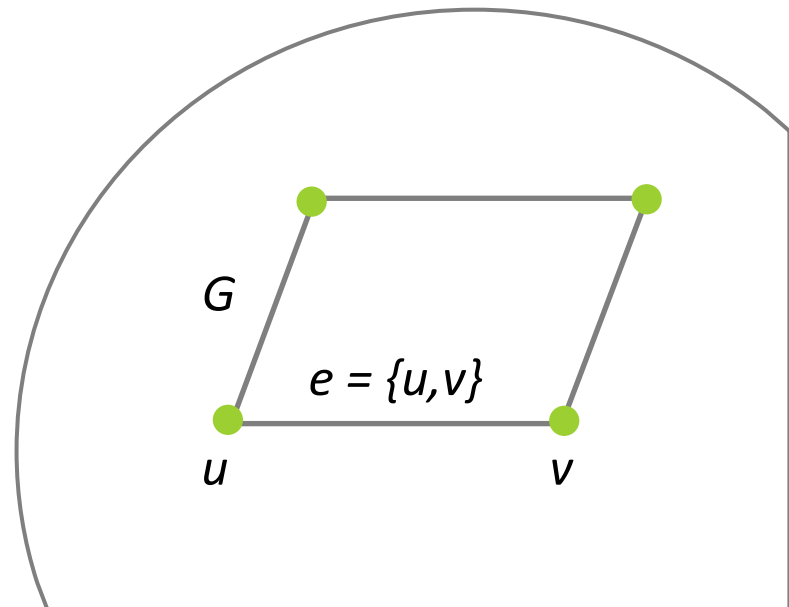
# Introduction to Graphs: Definition



A **graph**  $G = (V, E)$  is a structure consisting of a set  $V$  of vertices (nodes) and a set  $E$  of edges (lines joining vertices).

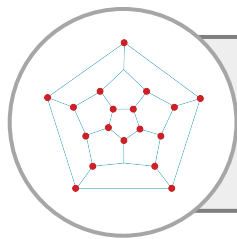
- Two vertices  $u$  and  $v$  are **adjacent** in  $G$  if  $\{u, v\}$  is an edge of  $G$ .
- If  $e = \{u, v\}$ , the edge  $e$  is called **incident** with the vertices  $u$  and  $v$ .

Graphs are useful to represent data.



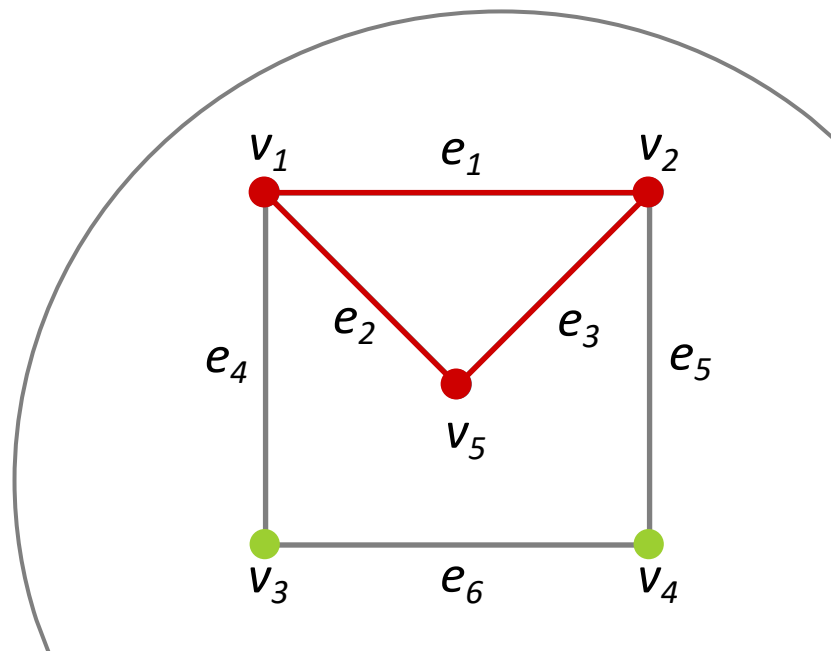
# Types of Graphs

# Types of Graphs: Subgraphs



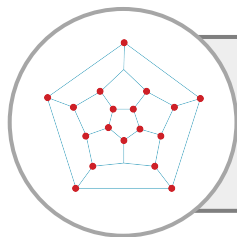
A graph  $H = (V_H, E_H)$  is a **subgraph** of  $G = (V_G, E_G)$  if  $V_H$  is a subset of  $V_G$  and  $E_H$  is a subset of  $E_G$ .

- $V_H = \{v_1, v_2, v_5\}$  is a subset of  $V_G$
- $E_H = \{e_1, e_2, e_3\}$  is a subset of  $E_G$





# Types of Graphs: Simple Graphs



A **simple** graph is a graph that has no **loop** (= edge  $\{u,v\}$  with  $u = v$ ) and no parallel edges between any pair of vertices.

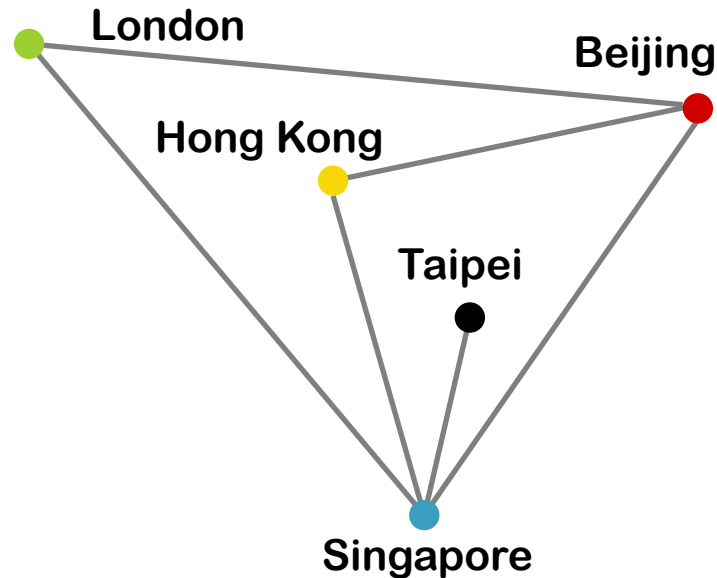
| From\To   | Hong Kong | Singapore | Beijing   | Taipei   | London   |
|-----------|-----------|-----------|-----------|----------|----------|
| Hong Kong |           | 4 Flights |           |          |          |
| Singapore | 2 Flights |           | 3 Flights | 1 Flight | 1 Flight |
| Beijing   | 1 Flight  | 2 Flights |           |          |          |
| Taipei    |           |           |           |          |          |
| London    |           | 1 Flight  | 1 Flight  |          | 1 Flight |

Draw a graph to see whether there are direct flights between any two cities (in either direction).

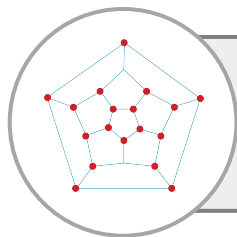
# Types of Graphs: Simple Graphs

| From\To   | Hong Kong | Singapore | Beijing   | Taipei   | London   |
|-----------|-----------|-----------|-----------|----------|----------|
| Hong Kong |           | 4 Flights |           |          |          |
| Singapore | 2 Flights |           | 3 Flights | 1 Flight | 1 Flight |
| Beijing   | 1 Flight  | 2 Flights |           |          |          |
| Taipei    |           |           |           |          |          |
| London    |           | 1 Flight  | 1 Flight  |          | 1 Flight |

Draw a graph to see whether there are direct flights between any two cities (in either direction).



# Types of Graphs: Multigraphs



A **multigraph** is a graph that has no loop and at least 2 parallel edges between some pair of vertices.

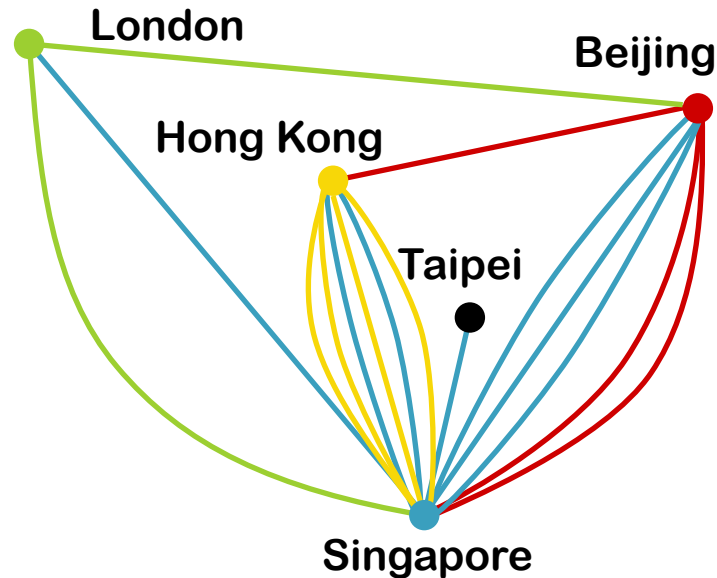
| From\To   | Hong Kong | Singapore | Beijing   | Taipei   | London   |
|-----------|-----------|-----------|-----------|----------|----------|
| Hong Kong |           | 4 Flights |           |          |          |
| Singapore | 2 Flights |           | 3 Flights | 1 Flight | 1 Flight |
| Beijing   | 1 Flight  | 2 Flights |           |          |          |
| Taipei    |           |           |           |          |          |
| London    |           | 1 Flight  | 1 Flight  |          | 1 Flight |

Draw a graph with an edge for each flight that operates between two cities (in either direction).

# Types of Graphs: Multigraphs

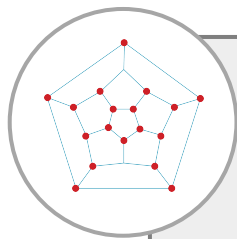
| From\To   | Hong Kong | Singapore | Beijing   | Taipei   | London   |
|-----------|-----------|-----------|-----------|----------|----------|
| Hong Kong |           | 4 Flights |           |          |          |
| Singapore | 2 Flights |           | 3 Flights | 1 Flight | 1 Flight |
| Beijing   | 1 Flight  | 2 Flights |           |          |          |
| Taipei    |           |           |           |          |          |
| London    |           | 1 Flight  | 1 Flight  |          | 1 Flight |

Draw a graph with an edge for each flight that operates between two cities (in either direction).





# Types of Graphs: Directed (Multi) Graphs



A **directed** graph is a graph where edges  $\{u,v\}$  are ordered, that is, edges have a direction. Parallel edges are allowed in **directed multigraphs**. Loops are allowed for both.

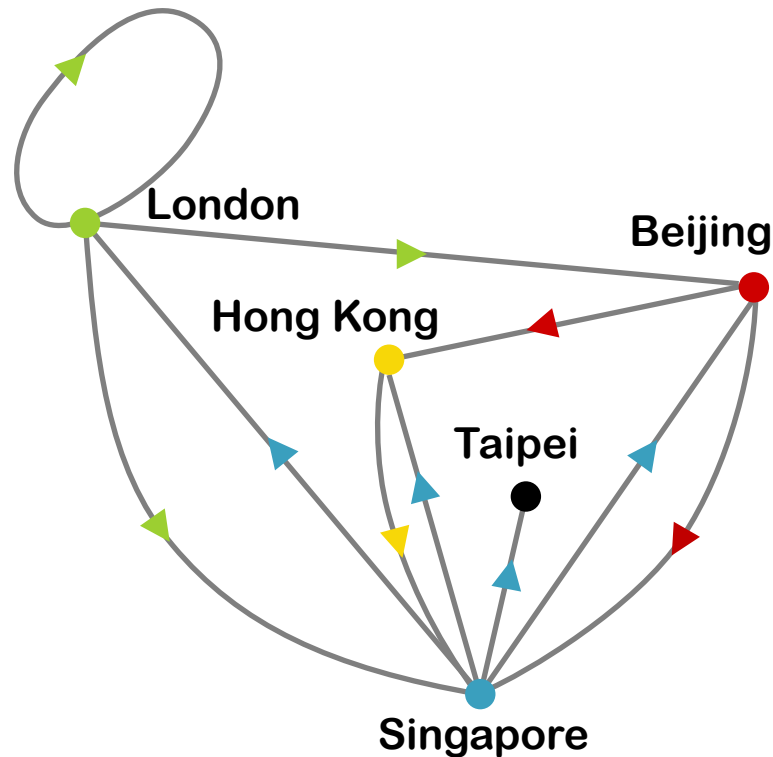
| From\To   | Hong Kong | Singapore | Beijing   | Taipei   | London   |
|-----------|-----------|-----------|-----------|----------|----------|
| Hong Kong |           | 4 Flights |           |          |          |
| Singapore | 2 Flights |           | 3 Flights | 1 Flight | 1 Flight |
| Beijing   | 1 Flight  | 2 Flights |           |          |          |
| Taipei    |           |           |           |          |          |
| London    |           | 1 Flight  | 1 Flight  |          | 1 Flight |

Draw a graph to see whether there are direct flights between any two cities (direction matters).

# Types of Graphs: Directed (Multi) Graphs

| From\To   | Hong Kong | Singapore | Beijing   | Taipei   | London   |
|-----------|-----------|-----------|-----------|----------|----------|
| Hong Kong |           | 4 Flights |           |          |          |
| Singapore | 2 Flights |           | 3 Flights | 1 Flight | 1 Flight |
| Beijing   | 1 Flight  | 2 Flights |           |          |          |
| Taipei    |           |           |           |          |          |
| London    |           | 1 Flight  | 1 Flight  |          | 1 Flight |

Draw a graph to see whether there are direct flights between any two cities (direction matters).



# Euler Theorem

# Euler Theorem: The Mathematician

Leonhard Euler introduced graphs in 1736 to solve the Königsberg Bridge problem.

What is the “Königsberg Bridge problem”?



**Kaliningrad (Königsberg)  
in Russia**



**Leonhard Euler  
1707 - 1783**

Portrait of Leonhard Euler by Jakob Emanuel Handmann under WikiCommons (PD-US)

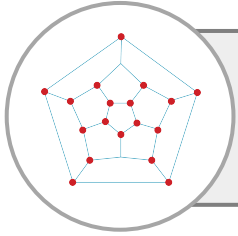
# Euler Theorem: Origin (Bridges of Königsberg)

- Königsberg (now known as Kaliningrad in Russia) has 7 bridges.
- People tried (without success) to find a way to walk all 7 bridges without crossing a bridge twice.
- Leonhard Euler proved that it was **impossible** to walk all seven bridges without crossing a bridge twice.



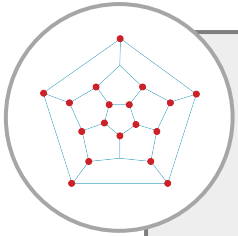
Seven Bridges of Königsberg

# Euler Circuit: Definitions



A **Euler path** (Eulerian trail) is a walk on the edges of a graph which uses each edge in the original graph exactly once.

The beginning and end of the walk may or may not be the same vertex.

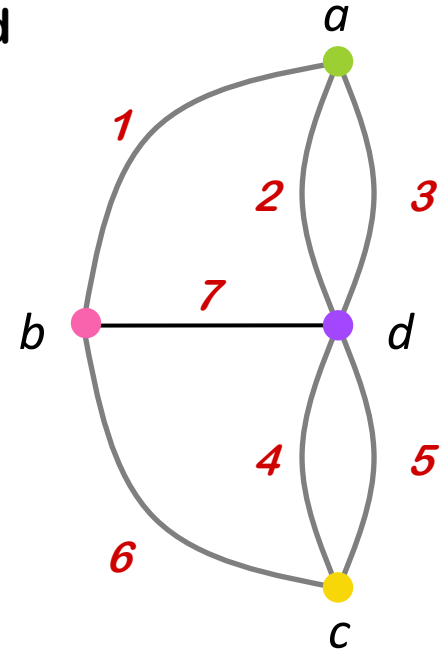
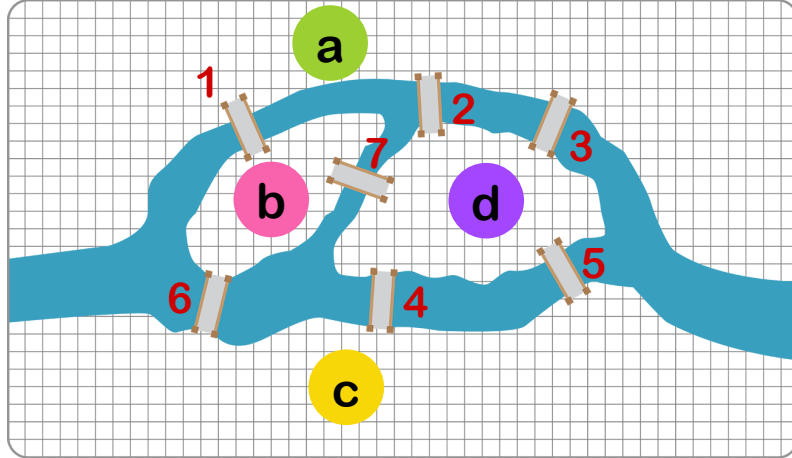


A **Euler circuit** (Eulerian cycle) is a walk on the edges of a graph which starts and ends at the same vertex, and uses each edge in the original graph exactly once.



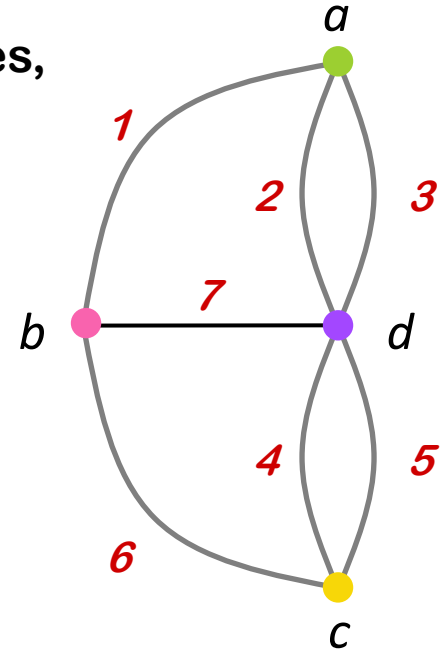
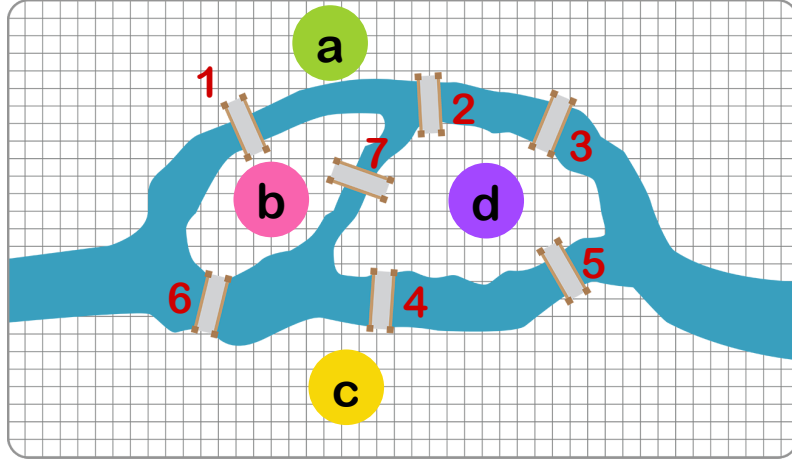
# Euler Circuit

- Suppose the beginning and end are the same node  $u$ .
- The graph must be **connected**.
- At every vertex  $v \neq u$ , we reach  $v$  along one edge and go out along another, thus the number of edges incident at  $v$  (called the degree of  $v$ ) is even.

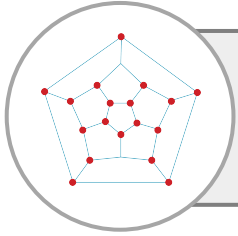


# Euler Circuit

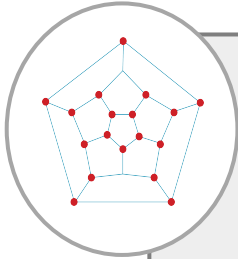
- The node  $u$  is visited once the first time we leave, and once the last time we arrive, and possibly in between (back and forth), thus the degree of  $u$  is even.
- Since the Königsberg Bridges graph has odd degrees, it has no solution!



# Euler Theorem



The **degree** of a vertex is the number of edges incident with it.



**Theorem:** consider a connected graph  $G$ .

1. If  $G$  contains an Euler path that starts and ends at the same node, then all nodes of  $G$  have an even degree.
2. If  $G$  contains an Euler path that starts and ends at two distinct nodes, then exactly two nodes of  $G$  have an odd degree.

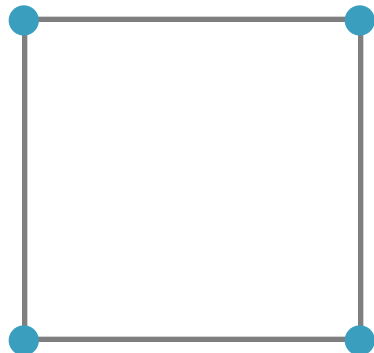
# Euler Theorem

---

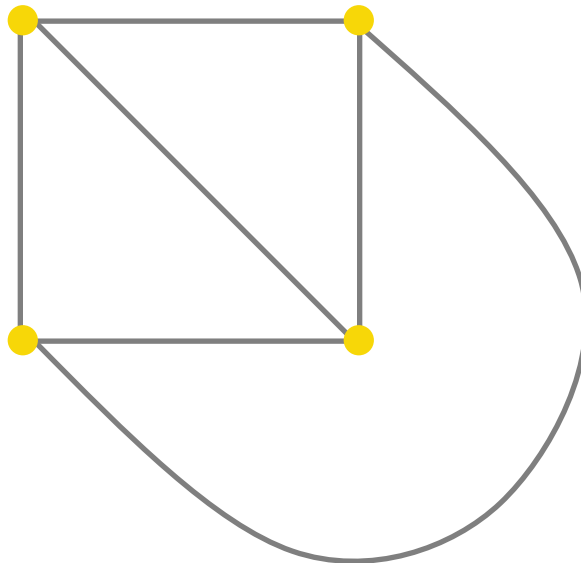
- Suppose  $G$  has an Euler path, which starts at  $v$  and finishes at  $w$ .
- Add the edge  $\{v, w\}$ .
- Then by the first part of the theorem, all nodes have even degrees, except for  $v$  and  $w$  which have odd degrees.

# Euler Theorem: Examples

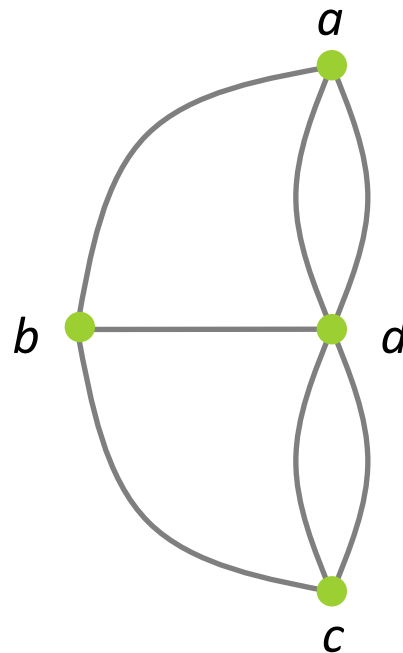
Note: Euler Theorem actually states an “if and only if” statement.



Euler Circuit



No Euler Path



No Euler Path

# Topic Summary



# Let's recap...

- Graph theory has numerous applications (e.g., networks, distributed systems, coding theory)
- Parts of a graph:
  - Vertex
  - Edge
  - Adjacent
  - Incident



# Let's recap...

---

- Types of graphs:
  - Simple graph
  - Multigraph
  - Directed (multi) graph
- Euler path, Euler circuit and Euler theorem

